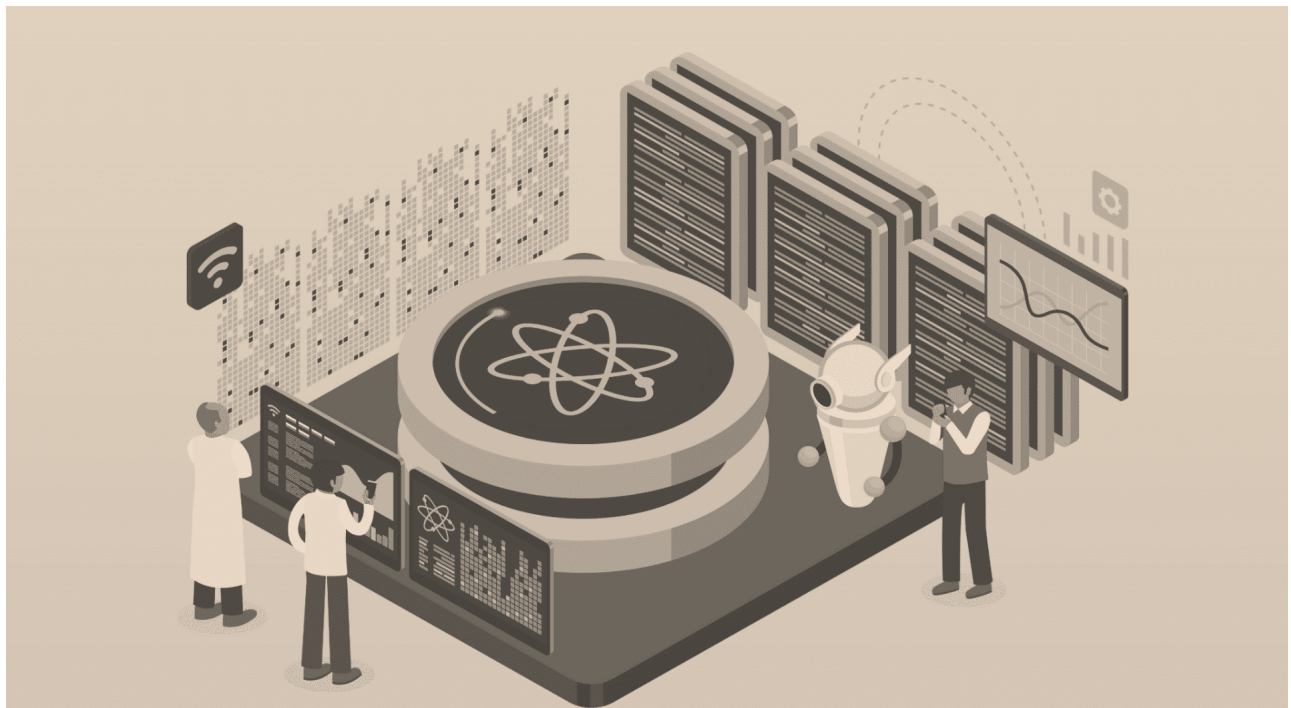


Designing a data stream pipeline for decision tree training

Using Kafka Prometheus, Grafana, Spark, machine learning algorithms, and Postgres database

Hamidreza Kordi



Iran University of Science and Technology

Pre-revenue

Big Data Project Document 1403

The project described in the following document was completed with the guidance of Supervisor, **Dr.Naderi** and Advisor ,**Mr.Keshvari** The right to use it is authorized and at the disposal of the Computer Department of Iran University of Science and Technology, with the source being mentioned.

HamidRezaKordi(student number: 403723951)

Autumn 1403

Table of Contents

<i>Introduction</i>	<i>4</i>
What is Big Data?.....	4
The Role of Big Data in Modern Enterprises	5
The Technology Behind Big Data	6
Challenges in Big Data Analytics.....	7
<i>Big Data Processing Pipeline: Kafka, Spark, PostgreSQL, and Prometheus-Grafana</i>	
<i>Monitoring</i>	<i>9</i>
Project Overview	9
Preparing the Big Picture	11
<i>Dataset</i>	<i>14</i>
About Dataset	14
<i>Data Source & Ingestion</i>	<i>15</i>
Data Source and Ingestion with Apache Kafka	15
Implementation.....	16
Kafka process in the designed pipeline	19
<i>Storage Layer (PostgreSQL & Docker)</i>	<i>32</i>
What is Docker?.....	32
What is PostgreSQL?	32
Database Initialization with Python (psycopg2).....	33
big picture of this section.....	36
<i>Model Training with Apache Spark</i>	<i>37</i>
What is Spark?.....	37
What is pyspark?	39
<i>Model Evaluation & Testing</i>	<i>41</i>
Implementation.....	41
Review.....	46
<i>Monitoring & Visualization with Prometheus & Grafana</i>	<i>48</i>
Grafana: An Overview.....	48
Prometheus: An Overview	48
Connecting Grafana and Prometheus	49
Below are some sample images from Grafana showing our user panels and charts.	57

Introduction

What is Big Data?

Big Data refers to large, complex sets of data that are difficult to process and analyze using traditional data processing techniques. It includes data that is generated from various sources in massive volumes and at high velocities, and it is often unstructured or semi-structured. The primary characteristics of Big Data are commonly referred to as the **Three Vs**: Volume, Velocity, and Variety.

- **Volume**: This refers to the sheer amount of data that is generated every day. With the proliferation of devices, social media platforms, sensors, and machines, the amount of data being produced is immense and continues to grow at an exponential rate.
- **Velocity**: Big Data is generated at high speeds and requires real-time processing. Social media, IoT devices, and financial markets generate vast amounts of data continuously, which must be captured, processed, and analyzed in real time to extract meaningful insights.
- **Variety**: Data comes in many forms, from structured data in relational databases to unstructured data such as text, images, videos, and social media posts. This wide variety of data types presents unique challenges for processing and analysis.

In recent years, **Big Data** has become a focal point for many organizations across different industries because of the enormous potential it offers in terms of insights and decision-making. Technologies and tools such as **Hadoop**, **Spark**, **NoSQL databases**, and cloud computing platforms have been developed to handle and analyze Big Data efficiently.

The Role of Big Data in Modern Enterprises

Big Data plays a transformative role in various industries such as finance, healthcare, retail, marketing, and logistics. By leveraging Big Data technologies, companies are able to gain deeper insights into customer behavior, improve decision-making, optimize operational efficiencies, and drive innovation.

Key Applications of Big Data:

1. Predictive Analytics:

Predictive analytics is one of the primary applications of Big Data. Organizations use Big Data to analyze historical data and make predictions about future events. For example, financial institutions use predictive models to detect fraudulent transactions, while retail businesses predict future sales trends based on consumer buying patterns.

2. Customer Insights:

Big Data allows companies to gather detailed customer data, including browsing behavior, purchasing patterns, and social media interactions. By analyzing this data, businesses can segment customers, personalize marketing efforts, and improve the overall customer experience. This helps in building stronger customer relationships and increasing customer loyalty.

3. Operations Optimization:

Companies can use Big Data to enhance their operations and improve efficiencies. For instance, logistics companies use real-time tracking data to optimize delivery routes, reduce fuel consumption, and minimize delivery times. Similarly, manufacturers use sensor data to monitor equipment performance, predict maintenance needs, and avoid costly downtime.

4. Healthcare:

In healthcare, Big Data is used to improve patient care by analyzing electronic health records (EHRs), medical imaging, genetic data, and real-time patient monitoring. By leveraging advanced analytics, healthcare professionals can provide more accurate diagnoses, personalize treatment plans, and even predict disease outbreaks.

5. Fraud Detection and Security:

Big Data plays a significant role in fraud detection and cybersecurity. Financial institutions and insurance companies use Big Data technologies to analyze transaction patterns and identify suspicious activity. In the field of cybersecurity, data analytics tools help detect network intrusions and protect sensitive information.

6. Social Media and Sentiment Analysis:

Social media platforms like Facebook, Twitter, and Instagram generate massive amounts of data. By analyzing this data, businesses can track brand sentiment, monitor customer feedback, and even predict public opinion on political or social issues. Social media analytics is also used to target specific user groups with personalized content and advertisements.

7. Smart Cities and IoT:

The Internet of Things (IoT) generates vast amounts of data from connected devices such as sensors, cameras, and smart meters. This data is used in smart city applications like traffic management, environmental monitoring, and energy usage optimization. Smart cities leverage Big Data to make urban environments more sustainable, efficient, and livable.

The Technology Behind Big Data

The enormous volume and complexity of Big Data have led to the development of specialized technologies and platforms to store, process, and analyze data efficiently. Some of the key technologies include:

1. Hadoop:

Apache Hadoop is one of the most widely used open-source Big Data frameworks. It provides a distributed computing environment where data is stored across a network of machines and processed in parallel. Hadoop's HDFS (Hadoop Distributed File System) allows for scalable, fault-tolerant storage, while its MapReduce programming model enables distributed data processing.

2. Apache Spark:

Apache Spark is a fast, in-memory processing engine for Big Data analytics. Unlike Hadoop MapReduce, Spark processes data in-memory, making it significantly faster for many data processing tasks. Spark supports a wide range of data processing capabilities, including batch processing, stream processing, machine learning, and graph processing.

3. NoSQL Databases:

Traditional relational databases are not well-suited for handling unstructured or semi-structured data at scale. NoSQL databases, such as MongoDB, Cassandra, and HBase, have emerged as alternatives to store and process Big Data. These databases offer scalability, flexibility, and high availability for handling large volumes of diverse data.

4. Cloud Computing:

Cloud platforms like Amazon Web Services (AWS), Microsoft Azure, and Google Cloud provide scalable infrastructure and storage solutions for Big Data. Cloud-based services enable organizations to process and analyze data without the need to invest in on-premises hardware. This has made Big Data more accessible to businesses of all sizes.

5. Data Lakes:

A data lake is a centralized repository that allows you to store structured and unstructured data at scale. Data lakes are built on distributed storage platforms like Hadoop or cloud storage solutions and provide a flexible environment for storing raw, untransformed data before it is processed or analyzed.

6. Machine Learning and AI:

Machine learning (ML) and artificial intelligence (AI) are integral components of Big Data analytics. By applying algorithms to large datasets, organizations can uncover patterns, trends, and insights that were previously impossible to detect. ML models can be trained on historical data and then deployed to make predictions on new, unseen data.

Challenges in Big Data Analytics

Despite its immense potential, working with Big Data comes with several challenges:

1. Data Quality:

The accuracy and reliability of data are crucial in Big Data analytics. Inconsistent, incomplete, or noisy data can lead to inaccurate analysis and poor decision-making. Cleaning and transforming raw data is often a time-consuming and complex process.

2. Data Security and Privacy:

As the volume of data increases, so do concerns about data security and privacy. Protecting sensitive data, complying with regulations (such as GDPR), and ensuring that data is used ethically are major challenges in the Big Data ecosystem.

3. Data Integration:

Big Data is often stored in various formats and across different systems, making it difficult to integrate and analyze. Integrating data from different sources (e.g., transactional databases, social media, IoT devices) requires advanced tools and techniques to ensure compatibility and coherence.

4. Scalability:

The sheer scale of Big Data requires distributed systems and computing power. Handling massive datasets efficiently and scaling analytics infrastructure as data grows can be a significant challenge for organizations.

5. Real-time Processing:

Real-time Big Data analytics is a key requirement for many applications, such as fraud detection, predictive maintenance, and IoT data analysis. Building systems that can process and analyze data in real-time requires specialized technologies and low-latency frameworks.

Big Data Processing Pipeline: Kafka, Spark, PostgreSQL, and Prometheus-Grafana Monitoring

Project Overview

This project is designed as a **Big Data processing pipeline** that efficiently handles large-scale structured data using a combination of **streaming, batch processing, and distributed computing** technologies. The core of this system revolves around **Apache Kafka**, which acts as the central messaging layer to facilitate real-time data movement between various components. The pipeline begins with a **large CSV dataset**, which is ingested into the system through Kafka, ensuring scalability and efficient data transmission.

Once the data is ingested, it is **stored in PostgreSQL**, which serves as an intermediate structured storage layer. However, rather than deploying a traditional database server, this project utilizes **Docker** to containerize PostgreSQL, ensuring easy deployment, scalability, and environment consistency. Running the database in a containerized environment enhances portability and simplifies the management of dependencies across different machines.

After storage, the data is extracted from PostgreSQL and, once again, passed through Kafka for further processing. This mechanism allows the system to distribute the data into **separate CSV files**, facilitating modular data handling. The **Apache Spark** engine then reads these individual CSV files and **trains machine learning models** on them. The system is designed in such a way that each dataset is processed separately, ensuring a structured and parallelized approach to model training. Additionally, Spark is also used to evaluate these models by testing them on a separate CSV file, allowing for validation before deployment.

Once the training process is completed, the **final trained models** are stored as files for later use. Simultaneously, the **test results** generated from the evaluation process are saved in a structured **JSON file**, which can later be used for analysis, reporting, or further optimizations.

A crucial aspect of this pipeline is its **observability and monitoring capabilities**. To track system performance and identify potential bottlenecks, **Prometheus** is integrated into the pipeline, collecting real-time metrics related to resource consumption, data processing times, and system load. These metrics are then visualized through **Grafana**, providing a comprehensive **dashboard for real-time monitoring**. This visualization tool helps analyze various performance aspects, such

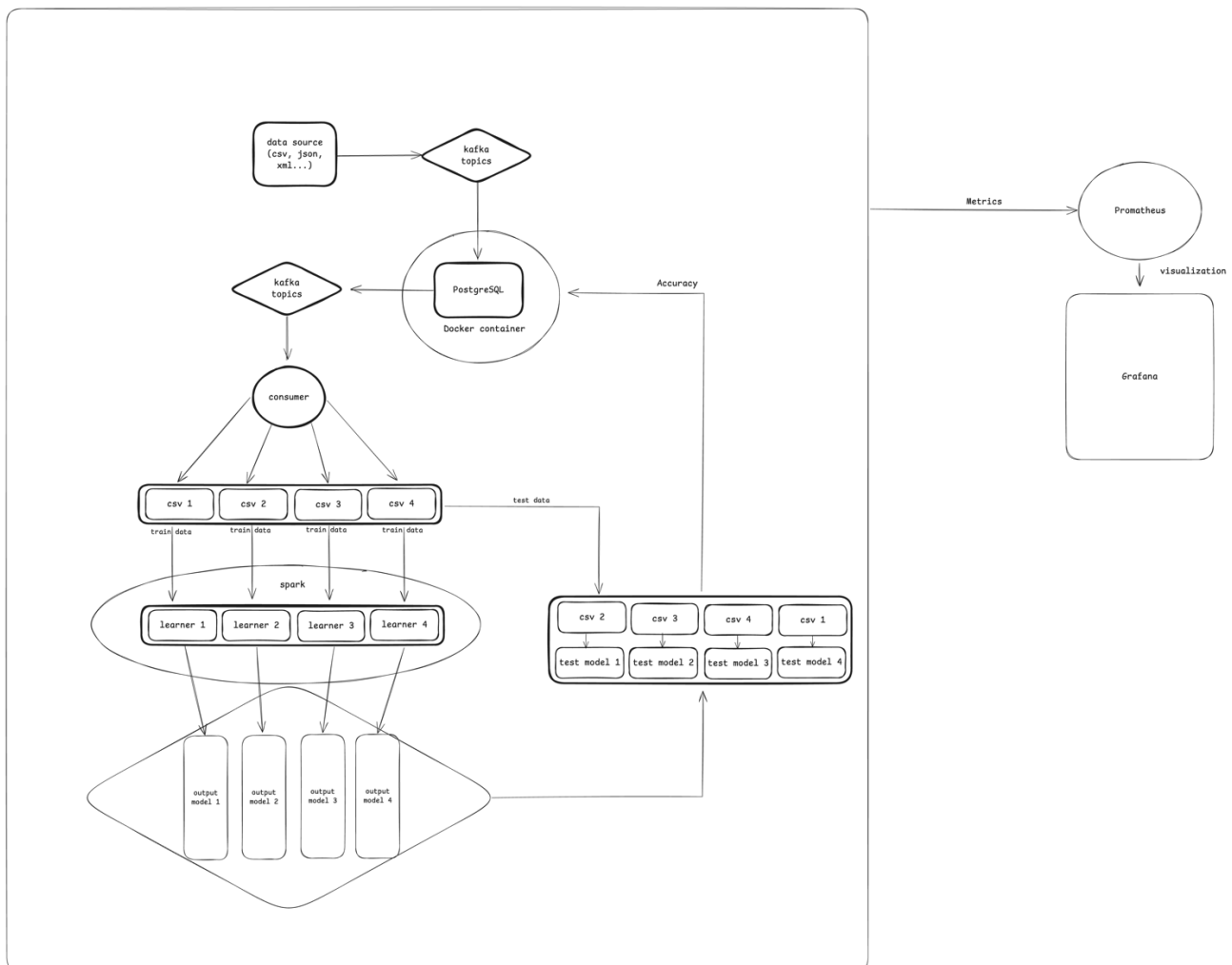
as Kafka lag, database query execution times, Spark job completion rates, and overall resource utilization.

By leveraging **Docker**, the entire system, including Kafka, PostgreSQL, Spark, Prometheus, and Grafana, can be deployed in an isolated and reproducible environment, eliminating configuration inconsistencies and simplifying infrastructure management. This approach makes the pipeline not only **scalable and modular** but also **fault-tolerant and highly adaptable** to different data processing scenarios.

In summary, this project seamlessly integrates **real-time streaming, batch processing, machine learning model training, structured data storage, and performance monitoring** into a unified **Big Data architecture**. Through the combined power of **Kafka, PostgreSQL, Spark, Docker, Prometheus, and Grafana**, this system provides a **robust, scalable, and efficient** framework for handling large-scale data with high performance and observability.

Preparing the Big Picture

This **Big Data Processing Pipeline Diagram** provides a visual representation of how data flows through different components, from ingestion to processing, model training, and monitoring. Below is a detailed breakdown of the architecture and its functionalities:



This **Big Data Processing Pipeline** is designed to handle large-scale structured data efficiently by integrating **streaming, batch processing, distributed computing, and monitoring** technologies. At its core, **Apache Kafka** is responsible for real-time data ingestion and movement between different components, ensuring a scalable and fault-tolerant workflow. The process begins with a **large CSV dataset**, which serves as the primary data source and is streamed into Kafka. This ensures that data can be consumed asynchronously by multiple processing units without performance bottlenecks.

Once the data is ingested, it is stored in **PostgreSQL**, which acts as an intermediate structured storage system. However, instead of running PostgreSQL on a dedicated database server, the system utilizes **Docker** to containerize the database. This approach enhances **scalability, portability, and ease of deployment** while ensuring a consistent runtime environment. By running the database in a container, the infrastructure remains modular and can be easily managed across different environments. PostgreSQL plays a critical role in **data persistence**, enabling structured storage, retrieval, and efficient querying before the next processing stage.

After being stored, the data is retrieved from PostgreSQL and pushed back into **Kafka topics**, ensuring a structured streaming process that allows multiple downstream consumers to access and process the data efficiently. This intermediate step is crucial as it enables the system to generate **separate CSV files**, facilitating modular processing and parallel execution. A **Kafka consumer** reads the data and ensures it is correctly formatted for the next phase of the pipeline.

Once the data is organized into CSV files, it is passed to **Apache Spark**, which serves as the main processing engine for machine learning model training. Spark processes each dataset separately, allowing for parallelized training workflows where different datasets are assigned to distinct learners. This design ensures that models are trained independently, preventing data overlap while maximizing efficiency in distributed computing. After the training phase, the system evaluates model performance using an additional **test dataset**, ensuring a structured validation process before deployment. The **final trained models** are stored in separate files for future inference, while the **test results** are written to structured **JSON files**, ensuring clear documentation of model accuracy and performance.

A key component of this architecture is its **monitoring and observability** capabilities. Throughout the data processing workflow, **Prometheus** is integrated to collect real-time system metrics such as **Kafka topic lag, PostgreSQL query execution times, Spark job durations, resource utilization, and model training times**. This ensures that every step of the pipeline is continuously tracked for performance optimization. The collected metrics are then sent to **Grafana**, where they are visualized in an interactive **dashboard** that provides insights into **data flow, processing efficiency, and system health**. By leveraging Prometheus and Grafana, the system can proactively detect and address performance bottlenecks, ensuring a highly **optimized and fault-tolerant** architecture.

By containerizing its core components with **Docker**, this pipeline ensures **scalability, consistency, and ease of deployment** across different environments. The combination of **Kafka, PostgreSQL, Spark, Docker, Prometheus, and Grafana** results in a **robust, modular, and scalable system** that can efficiently process large volumes of data while maintaining real-time monitoring and performance optimization. This architecture allows for seamless **data ingestion, transformation, model training, testing, and visualization**, making it highly suitable for **large-scale real-time analytics and machine learning applications**.

Dataset

About Dataset

This project is about predicting if a flight will be delayed by over 15 minutes upon arrival, with Decision Tree Classifier, using US flight data in 2022.

The U.S. Department of Transportation's (DOT) Bureau of Transportation Statistics tracks the on-time performance of domestic flights operated by large air carriers. This dataset is collected from the Bureau of Transportation Statistics, Govt. of the USA. This data is open-sourced under U.S. Govt. Works.

Description of Columns

- Quarter Quarter (1-4)
- Month Month
- Dayo Month Day of Month
- DayOfWeek Day of Week
- FlightDate Date of the Flight
- Marketing_Airline_Network Airline Identifier
- OriginCityName Origin Airport, City Name
- DestCityName Destination Airport, City Name
- DepDelay Difference in minutes between scheduled and actual departure time. Early departures show negative numbers
- ArrDelay Difference in minutes between scheduled and actual arrival time. Early arrivals show negative numbers
- Cancelled Cancelled Flight (1=Yes)
- Diverted Diverted Flight (1=Yes)
- AirTime Flight Time, in Minutes
- Distance Distance between airports (miles)
- CarrierDelay Delay caused by the airline in minutes
- WeatherDelay Delay caused by weather
- NASDelay Delay caused by air system
- SecurityDelay Delay caused by security reasons
- LateAircraftDelay Delay caused as a result of another flight on the same aircraft delayed

Data Source & Ingestion

Data Source and Ingestion with Apache Kafka

In a **Big Data processing pipeline**, the first and most critical step is **data ingestion**, where raw data is collected from various sources and made available for further processing. This stage ensures that the system can handle high-throughput data efficiently while maintaining scalability and fault tolerance. **Apache Kafka**, a **distributed event streaming platform**, plays a crucial role in this process by providing a **scalable, reliable, and fault-tolerant** way to ingest and distribute data across multiple consumers.

The **data source** in a Kafka-based ingestion system can come from various formats, including **CSV files, JSON data, XML feeds, logs, IoT sensor data, databases, API endpoints, or real-time event streams**. These sources generate large amounts of structured or unstructured data, which needs to be processed efficiently to derive insights. Instead of sending this data directly to a database or processing engine, Kafka acts as an intermediary, ensuring **decoupling between data producers and consumers** while maintaining high availability.

Kafka organizes incoming data into **topics**, which serve as **logical channels** where events are produced and consumed. Each topic can have multiple **partitions**, allowing data to be distributed across multiple nodes for parallel processing. This partitioning ensures that Kafka can scale horizontally, handling large volumes of incoming messages without performance degradation.

When data enters Kafka, it is produced by **producers**, which are responsible for **publishing** data into the appropriate topic. Producers can be anything from **log forwarders, ETL pipelines, data streams from IoT devices, database change logs, or real-time applications**. These producers send data asynchronously, ensuring minimal delay and efficient buffering of high-volume streams.

Once data is inside Kafka, it remains available in its distributed storage system, where **consumers** can read and process it. Consumers can be **batch processing systems, real-time analytics engines, machine learning pipelines, or storage layers like PostgreSQL, Hadoop, or NoSQL databases**. Since Kafka supports a **publish-subscribe model**, multiple consumers can read the same data stream simultaneously, allowing different parts of the system to process data independently.

Kafka also provides **fault tolerance and durability** by replicating messages across multiple broker nodes. This means that even if a broker fails, data remains available, ensuring **zero data loss**. The distributed nature of Kafka allows it to handle **massive data ingestion workloads** efficiently, making it an ideal choice for real-time **log aggregation, monitoring, fraud detection, and data pipelines**.

In a **containerized environment**, Kafka is often deployed using **Docker or Kubernetes**, which further enhances its **scalability, orchestration, and fault tolerance**. By using **Kafka Connect**, external data sources such as relational databases (PostgreSQL, MySQL), cloud storage (AWS S3, Google Cloud Storage), or enterprise data warehouses can be seamlessly integrated into the Kafka ecosystem.

Implementation

The following steps will show the details of the project to use Kafka. In this project, Kafka is used in three places: In the first stage, it is used to dump data into the database, in the second stage, it enters the data into the CSV creation flow and creates the files, and in the last stage, it stores the test results in the database. This guide will walk you through installing and running **Apache Kafka** on **Ubuntu**. Follow each step carefully to ensure a successful setup.

Step 1: Update System Packages

Before installing Kafka, update your system's package list to ensure you have the latest versions of available packages.

```
>> sudo apt update && sudo apt upgrade -y
```

Step 2: Install Java (JDK)

Kafka requires **Java** to run, so you need to install **OpenJDK** if it is not already installed.

```
>> sudo apt install openjdk-11-jdk -y
```

Check the installed Java version:

```
>> java -version
```


Step 3: Download and Extract Apache Kafka

Download the latest stable version of Kafka from the official Apache website.

```
>> cd /opt
```

```
>> sudo wget https://downloads.apache.org/kafka/3.5.1/kafka\_2.13-3.5.1.tgz
```

Now extract the downloaded file:

```
>> sudo tar -xvzf kafka_2.13-3.5.1.tgz
```

Rename the folder for easier access:

```
>> sudo mv kafka_2.13-3.5.1 kafka
```

Step 4: Start Zookeeper (Kafka Dependency)

Kafka requires **Zookeeper** to manage its metadata. First, start **Zookeeper** before running Kafka.

Navigate to the Kafka directory:

```
>> cd /opt/kafka
```

Start Zookeeper using the default configuration:

```
>> bin/zookeeper-server-start.sh config/zookeeper.properties
```

Zookeeper will now start and listen on port **2181**.

To run Zookeeper in the background, use:

```
>> bin/zookeeper-server-start.sh config/zookeeper.properties >  
/dev/null 2>&1 &
```

Step 5: Start Kafka Broker

After starting Zookeeper, you can now start the Kafka broker.

```
>> bin/kafka-server-start.sh config/server.properties
```

Kafka should now be running on **port 9092**.

To run Kafka in the background, use:

```
>> bin/kafka-server-start.sh config/server.properties > /dev/null 2>&1  
&
```

Step 6: Verify Kafka is Running

To ensure Kafka is running correctly, check the active processes:

```
>> ps aux | grep kafka
```

You should see an active Kafka process.

Additionally, check the open ports:

```
>> netstat -tulnp | grep :9092
```

If Kafka is running, it should show an active connection on port **9092**.

Step 7: Create a Kafka Topic

Kafka works with **topics**, so let's create one, we need three topics:

```
>> bin/kafka-topics.sh --create --topic topic_A --bootstrap-server  
localhost:9092 --partitions 1 --replication-factor 1
```

```
>> bin/kafka-topics.sh --create --topic topic_B --bootstrap-server  
localhost:9092 --partitions 1 --replication-factor 1
```

```
>> bin/kafka-topics.sh --create --topic topic_M --bootstrap-server  
localhost:9092 --partitions 1 --replication-factor 1
```

Verify that the topic was created successfully:

```
>> bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

installed and configured **Apache Kafka** on Ubuntu. Kafka is now fully operational with a topic created, messages being produced and consumed, and optional **systemd services** for automatic startup. This setup provides a **scalable, fault-tolerant, and high-throughput** data streaming environment.

Kafka process in the designed pipeline

Next, we will explain the details of the Kafka process in the designed pipeline.

Part 1: Reading data from the data source and sending it to the database

In the first stage, data is entered into the Kafka queue from the data source, and Kafka produces it and enters it into our desired topic, the code of which is as follows:

1_1. Importing Required Libraries

```
from kafka import KafkaProducer
import json
import csv
```

- **KafkaProducer**: This is imported from the `kafka-python` library and is used to send messages to Apache Kafka.
- **json**: Used for converting data into JSON format before sending it to Kafka.
- **csv**: Helps in reading data from a CSV file.

1_2. Kafka Configuration

```
KAFKA_BROKER = 'localhost:9092'
KAFKA_TOPIC = 'topic_A'
```

- **KAFKA_BROKER**: Defines the Kafka server address, which in this case is running locally at `localhost:9092`.
- **KAFKA_TOPIC**: Specifies the default topic where messages will be sent.

1_3. Topic Selection Function

```
def select_topic(num):  
    return "topic_A"
```

- This function is intended to determine the Kafka topic dynamically based on the input `num`.
- However, currently, it always returns `"topic_A"`, meaning all messages are sent to the same topic.

1_4. Initializing Kafka Producer

```
producer = KafkaProducer(  
    bootstrap_servers=KAFKA_BROKER,  
    value_serializer=lambda v: json.dumps(v).encode('utf-8')  
)
```

- `bootstrap_servers`: Specifies the Kafka broker the producer should connect to.
- `value_serializer`: Converts Python dictionaries into JSON format and encodes them in UTF-8, ensuring that messages are properly formatted before being sent.

1_5. Reading CSV File and Sending Messages

```
csv_file_path = 'df_EDA.csv'  
count = 0
```

- `csv_file_path`: Defines the path of the CSV file (`df_EDA.csv`) that contains the data to be sent.
- `count`: A counter variable to track the number of messages being sent.

1_6. Opening the CSV File and Processing Each Row and Sending Messages to Kafka

```
with open(csv_file_path, 'r') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        message = {key: row[key] for key in row}
        print(message)

        # Record the start time of sending the message
        with MESSAGE_LATENCY.time():
            producer.send(topic="topic_A", value=message)

MESSAGE_COUNT.inc() # Increment message count for the specific topic
count += 1

# Close the Kafka producer when done
producer.close()
```

- Opens the CSV file in read mode ('r').
- Uses `csv.DictReader` to read the CSV content as dictionaries, where each column name serves as a dictionary key.
- Iterates through each row, creating a dictionary (`message`) where keys are column headers, and values are row values.
- Prints the message to the console for debugging.
- `producer.send(...)`: Sends the message to Kafka.
 - The topic is selected dynamically via `select_topic(count)`, which currently always returns "topic_A".
 - `value=message` sends the dictionary (converted to JSON format) as the message payload.

Part 2: Reading data from the topic and save to database

In this step, Kafka reads the data from the topic and then dumps it into the database. Below we will refer to the code step by step.

2_1. Importing Required Libraries

```
from kafka import KafkaConsumer
import json
import logging
import psycopg2
```

- KafkaConsumer: Reads messages from a Kafka topic.
- json: Converts message data from JSON format.
- logging: Logs errors and execution details.
- psycopg2: Connects to a PostgreSQL database

2_2. Database Configuration

```
db_config = {
    "host": "localhost",
    "database": "mydb",
    "user": "hamid",
    "password": "1234",
    "port": 5432
}
```

- Stores PostgreSQL connection details, including host, database name, user, password, and port.
- WARNING: Hardcoding passwords is not secure—this should be stored in environment variables or a configuration file.

2_3. Batch Data Insertion Function

```
def insert_data_batch(cursor, data_batch):
    insert_query = """
    INSERT INTO FlightDetails(Index, Quarter, Month, DayOfMonth, DayOfWeek,
    FlightDate, Airlines, OriginCityName, DestCityName, DepDelay, ArrDelay, AirTime,
    Distance, Month_Str, DayOfWeek_Str, Flight_Status)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s);
    """
    for data in data_batch:
        cursor.execute(insert_query, data)
```

- Takes a batch of messages and inserts them into the `FlightDetails` table.
- Uses parameterized queries (`%s`) to prevent SQL injection.
- Loops through `data_batch` and executes an SQL `INSERT` statement for each record

2_4. Database Connection Function

```
def connect_to_db():
    return psycopg2.connect(**db_config)
```

- Uses `psycopg2.connect()` to establish a connection to PostgreSQL.
- The `**db_config` unpacks the dictionary values into keyword arguments for `psycopg2.connect()`.

2_5. Initializing Kafka Consumer

```
consumer = KafkaConsumer(
    'topic_A',
    bootstrap_servers='localhost:9092',
    group_id='my-group',
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
    auto_offset_reset='earliest'
)
```

- Connects to Kafka Broker (`localhost:9092`) and subscribes to `'topic_A'`.
- `group_id='my-group'`: Assigns the consumer to a consumer group (`my-group`), allowing multiple consumers to share the workload.
- `value_deserializer`: Converts Kafka messages from JSON format.
- `auto_offset_reset='earliest'`: If no previous offset exists, start consuming messages from the beginning.

2_6. Main Function - Kafka Message Processing

```
def main():
    conn = connect_to_db()
    cursor = conn.cursor()
    data_batch = []
```

- Connects to PostgreSQL (`conn = connect_to_db()`).
- Creates a database cursor (`cursor = conn.cursor()`) for executing SQL queries.

- Initializes an empty list (`data_batch`) for storing messages before batch insertion.

2_7. Processing Messages from Kafka

```
try:
    for message in consumer:
        # Start measuring message processing time
        with MESSAGE_PROCESSING_LATENCY.time():
            data = message.value
```

- Starts consuming messages from Kafka.
- Tracks processing time using Prometheus (`MESSAGE_PROCESSING_LATENCY.time()`).
- Extracts data fields from the JSON message and appends them as a tuple into `data_batch`

2_8. Batch Insertion Condition

```
data = message.value
if len(data_batch) >= 5:
    insert_data_batch(cursor, data_batch)
    conn.commit()
    data_batch.clear()
```

- Checks if `data_batch` contains at least 5 messages.
- If true:
 1. Inserts messages into PostgreSQL (`insert_data_batch()`).
 2. Commits the transaction (`conn.commit()`).
 3. Clears `data_batch` to prepare for new messages.
 4. Increments the batch insert counter (`INSERT_BATCH_COUNT.inc()`) for monitoring.

2_9. Error Handling

```
except Exception as e:
    logging.error(f"Error in Kafka consumer: {e}")
```

- Logs errors if an exception occurs.
- Increments an error counter (`ERROR_COUNT.inc()`) to track failures.

2_10. Graceful Shutdown

```
finally:
    logging.info("Shutting down Kafka consumer.")
    consumer.close()
    cursor.close()
    conn.close()
```

Part 3 : Send data from database to Divider

In the next step, we take the data stored in the database and import it into topic, which are then entered into Kafka for segmentation into separate CIV files.

3_1. Importing Required Libraries

```
import psycopg2
import json
from kafka import KafkaProducer
import time
```

- psycopg2: Establishes a connection to PostgreSQL.
- json: Converts database rows into JSON format.
- KafkaProducer: Sends data to Kafka.
- time: Introduces delays (`sleep`) between data fetches.

3_2. Connecting to PostgreSQL

```
# Database connection
conn = psycopg2.connect(
    host="localhost",
    database="mydb",
    user="hamid",
    password="1234",
    port=5432
)
cursor = conn.cursor()
```

- Creates a connection (`conn`) to PostgreSQL.
- Initializes a cursor (`cursor`) to execute queries.

- WARNING: Hardcoding credentials (`password="1234"`) is insecure; use environment variables

3_3. Setting Up Kafka Producer

```
producer = KafkaProducer(  
    bootstrap_servers='localhost:9092',  
    value_serializer=lambda v: json.dumps(v).encode('utf-8')  
)
```

- `bootstrap_servers='localhost:9092'`: Connects to the Kafka broker.
- `value_serializer`:
 - Converts messages to JSON using `json.dumps()`.
 - Encodes them as UTF-8 before sending.

3_4. Defining Batch Processing Variables

```
offset = 0  
batch_size = 1000
```

- `offset`: Keeps track of the last fetched row.
- `batch_size`: Determines how many rows are retrieved per batch (1000 rows at a time).

3_5. Function to Fetch and Send Data to Kafka

```
def fetch_and_send_data():  
    global offset  
    while True:
```

- Defines `fetch_and_send_data()` to process data in an infinite loop.
- Declares `offset` as `global`, so it retains its value across iterations.
- Runs indefinitely (`while True`) until no more data is available.

3_6. Fetching Data from PostgreSQL

```
with BATCH_PROCESSING_LATENCY.time():
    for row in rows:
        csv_data = {
            "Index": row[0],
            "Quarter": row[1],
            "Month": row[2],
            "DayOfMonth": row[3],
            "DayOfWeek": row[4],
            "FlightDate": str(row[5]),
            "Airlines": row[6],
            "OriginCityName": row[7],
            "DestCityName": row[8],
            "DepDelay": row[9],
            "ArrDelay": row[10],
            "AirTime": row[11],
            "Distance": row[12],
            "Month_Str": row[13],
            "DayOfWeek_Str": row[14],
            "Flight_Status": row[15]
        }
```

- Constructs a query to fetch `batch_size` rows (1000 rows).
- Uses `OFFSET {offset}` to avoid reprocessing old records.
- Executes the query (`cursor.execute(select_query)`).
- Fetches the results (`rows = cursor.fetchall()`).

3_7. Sending Data to Kafka

```
producer.send('topic_M', value=csv_data)
print(f"Sent data: {csv_data}")
```

- Sends `csv_data` to Kafka topic `'topic_M'`.
- Prints the sent message for debugging.

3_8. Updating Offset for Next Batch

```
offset += batch_size
time.sleep(1)
```

- Increases offset by batch_size (1000) to fetch the next set of rows.
- Waits 1 second (time.sleep(1)) before fetching the next batch

3_9. Running the Function

```
if __name__ == "__main__":
    fetch_and_send_data()
    cursor.close()
    conn.close()
    producer.close()
```

- Runs fetch_and_send_data() when the script starts.
- Closes all resources (cursor, conn, producer) when execution ends.

Part 4 : splite data to 4 csv splited

4_1. Importing Required Libraries

```
import csv
from kafka import KafkaConsumer
import json
```

- csv: Used for writing Kafka messages into CSV files.
- KafkaConsumer: Reads messages from Kafka topics.
- json: Deserializes Kafka messages from JSON format.

4_2. Kafka Consumer Setup

```
# Kafka Consumer Setup
consumer = KafkaConsumer(
    'topic_M',
    bootstrap_servers='localhost:9092',
    group_id='my-group',
```

```

    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
    auto_offset_reset='earliest',
)

```

- Subscribes to Kafka topic 'topic_M'.
- Connects to Kafka broker at 'localhost:9092'.
- Uses group_id='my-group', enabling multiple consumers to share the workload.
- Deserializes JSON messages (value_deserializer) before processing.
- Starts consuming from the earliest offset (auto_offset_reset='earliest'), ensuring no messages are missed.

4_3. Defining CSV File Paths

```

# File paths
csv_files = [
    "/home/hamidreza/Documents/bigdata/data/csv_file1.csv",
    "/home/hamidreza/Documents/bigdata/data/csv_file2.csv",
    "/home/hamidreza/Documents/bigdata/data/csv_file3.csv",
    "/home/hamidreza/Documents/bigdata/data/csv_file4.csv"
]

```

- Defines a list of CSV file paths where Kafka messages will be stored.
- Uses multiple files to distribute the data and prevent a single file from growing too large.

4_4. Defining CSV Headers

```

headers = [
    "Index", "Quarter", "Month", "DayOfMonth", "DayOfWeek", "FlightDate", "Airlines",
    "OriginCityName", "DestCityName", "DepDelay", "ArrDelay", "AirTime", "Distance",
    "Month_Str", "DayOfWeek_Str", "Flight_Status"
]

```

- Specifies the column names that will be written to each CSV file.
- Ensures consistency in the structure of stored data.

4_5.Ensuring CSV Headers Exist

```
# Ensure CSV headers exist
for path in csv_files:
    try:
        with open(path, 'a', newline='', encoding='utf-8') as file:
            writer = csv.writer(file)
            if file.tell() == 0:
                writer.writerow(headers)
    except Exception as e:
        print(f"Error initializing file {path}: {e}")
```

- Opens each CSV file in append mode ('a').
- Checks if the file is empty (`file.tell() == 0`):
If empty, writes the headers to maintain column consistency.
- Handles exceptions (e.g., file permission errors) and logs them.

4_6. Kafka Message Processing

```
# Process Kafka messages
count = 0
try:
    for message in consumer:
        data = message.value
        count += 1
```

- Initializes `count = 0` to track the number of processed messages.
- Iterates through Kafka messages (`for message in consumer`).
- Extracts the message value (`message.value`), which contains flight data.

4_7. Extracting Data and Writing to CSV

```
row = [data.get(header) for header in headers]
```

- Creates a list `row` that contains values for each column in `headers`.
- Ensures correct ordering of data fields before writing to CSV.

4_8. Writing the Data to CSV

```
file_index = count % len(csv_files)
with open(csv_files[file_index], 'a', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(row)
    print(f"Added to {csv_files[file_index]}")
```

- Opens the selected CSV file (`csv_files[file_index]`) in append mode (`'a'`).
- Writes the extracted row (`writer.writerow(row)`).
- Prints the file name to indicate successful writing.

4_9. Closing Kafka Consumer

```
except KeyboardInterrupt:
    print("Consumer stopped manually")
except Exception as e:
    print(f"Error while consuming Kafka messages: {e}")
finally:
    consumer.close()
```

Storage Layer (PostgreSQL & Docker)

What is Docker?

Docker is a **containerization platform** that allows developers to package applications and their dependencies into lightweight, portable containers. These containers ensure that the application runs consistently across different environments, eliminating issues related to dependency conflicts.

Key Advantages of Docker:

Portability – Applications can run on any system with Docker installed.

Isolation – Each container runs independently, preventing conflicts.

Scalability – Easily deploy and manage multiple instances of an application.

Efficiency – Uses system resources optimally, reducing overhead compared to virtual machines.

What is PostgreSQL?

PostgreSQL (often called **Postgres**) is a **powerful open-source relational database management system (RDBMS)**. It is widely used for handling large-scale applications and supports **ACID compliance, transactions, JSON, and advanced indexing techniques**. Why Use PostgreSQL with Docker?

Running PostgreSQL inside a Docker container provides several benefits:

- **Easy Setup:** Quickly deploy a PostgreSQL instance without manual installation.
- **Consistency:** Ensures the database runs in the same environment across development, testing, and production.
- **Backup & Restore:** Easily manage database snapshots and migrations.
- **Scalability:** Can be scaled across multiple containers or Kubernetes clusters.

Database Initialization with Python (psycopg2)

We use **Python** and the `psycopg2` library to **connect to the PostgreSQL database and create necessary tables**. Below is a step-by-step breakdown of the provided code:

Step 1: Define Database Configuration

```
import psycopg2

# Database configuration
db_config = {
    "host": "localhost",
    "database": "mydb",
    "user": "hamid",
    "password": "1234",
    "port": 5432
}
```

- Stores PostgreSQL connection details in a dictionary (`db_config`).
- Since PostgreSQL is running inside a Docker container, the `host="localhost"` refers to the Docker network bridge.

Step 2: Establish Connection to PostgreSQL

```
try:
    # Establish connection
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()
```

- Attempts to connect to the PostgreSQL database using `psycopg2.connect()`.
- Creates a cursor (`cursor = conn.cursor()`), which allows executing SQL queries.

Step 3: Create the `FlightDetails` Table

```
create_table_query = """
CREATE TABLE FlightDetails (
    Index VARCHAR(256),
    Quarter VARCHAR(256),
    Month VARCHAR(256),
    DayOfMonth VARCHAR(256),
    DayOfWeek VARCHAR(256),
    FlightDate DATE,
    Airlines VARCHAR(256),
    OriginCityName VARCHAR(256),
    DestCityName VARCHAR(256),
    DepDelay DOUBLE PRECISION,
    ArrDelay DOUBLE PRECISION,
    AirTime DOUBLE PRECISION,
    Distance DOUBLE PRECISION,
    Month_Str VARCHAR(256),
    DayOfWeek_Str VARCHAR(256),
    Flight_Status VARCHAR(256)
);
"""

cursor.execute(create_table_query)
conn.commit()
```

- Defines an SQL `CREATE TABLE` statement to create `FlightDetails`, which stores flight-related data.
- Uses appropriate data types:
 - `VARCHAR(256)`: Used for textual data (e.g., city names, airline names).
 - `DATE`: Used for the flight date.
 - `DOUBLE PRECISION`: Stores numerical values (e.g., delay times, distances).
- Executes the SQL query (`cursor.execute()`) to create the table.
- Commits the transaction (`conn.commit()`) to save changes.

Step 4: Create Multiple Tables for Accuracy Data

```
# Create accuracy_data table template
create_result_table_query = """
CREATE TABLE accuracy_data_model (
    id SERIAL PRIMARY KEY,
    rmse DOUBLE PRECISION NOT NULL,
    timestamp BIGINT NOT NULL,
    time TIMESTAMP NOT NULL
);
"""

for i in range(1, 5):
    table_name = f"accuracy_data_model{i}"
    cursor.execute(create_result_table_query.replace("accuracy_data_model",
table_name))
    conn.commit()
```

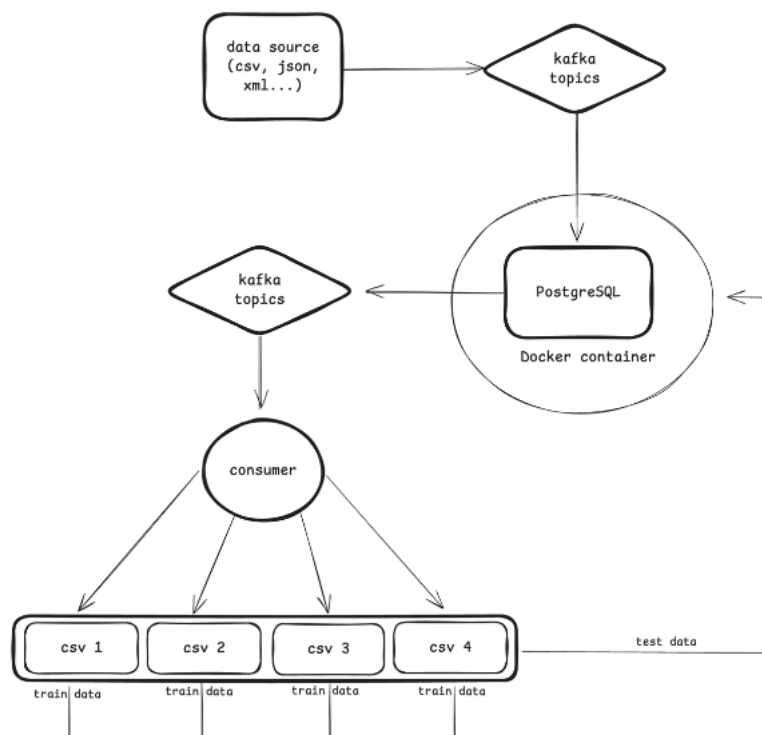
Defines a template SQL query for an `accuracy_data_model` table:

- `id SERIAL PRIMARY KEY`: An auto-incrementing ID.
 - `model_accuracy DOUBLE PRECISION NOT NULL`: Stores model accuracy values.
 - `timestamp BIGINT NOT NULL`: Stores a timestamp in UNIX format.
 - `time TIMESTAMP NOT NULL`: Stores a human-readable timestamp.
- Loops from 1 to 4 to create four separate tables:
 - `accuracy_data_model1`
 - `accuracy_data_model2`
 - `accuracy_data_model3`
 - `accuracy_data_model4`
 - Replaces `"accuracy_data_model"` with the dynamically generated table name (`table_name`).
 - Executes the query (`cursor.execute()`) to create each table.
 - Commits each transaction (`conn.commit()`).

Step 8: Closing the Database Connection

```
except Exception as e:
    print(f"Error: {e}")
finally:
    if conn:
        cursor.close()
        conn.close()
```

big picture of this section



Model Training with Apache Spark

What is Spark?

Apache Spark is an advanced, open-source distributed computing framework designed to handle massive datasets efficiently. Unlike traditional data processing systems, Spark is built to process large-scale data both in memory and on disk, making it significantly faster than Hadoop's MapReduce framework. It provides a unified ecosystem for various data processing needs, such as batch processing, real-time streaming, SQL querying, graph computation, and machine learning, all within a single framework.

At its core, Apache Spark follows a distributed data structure called Resilient Distributed Datasets (RDDs). These RDDs allow parallelized operations on data across multiple nodes in a cluster, ensuring fault tolerance and efficiency. Data is processed in-memory whenever possible, significantly reducing the need for disk I/O operations, which are often the primary bottleneck in big data applications. Furthermore, Spark is designed to run on multiple cluster management frameworks, including Hadoop YARN, Apache Mesos, and Kubernetes, and it also functions independently in a standalone mode. This flexibility allows Spark to be deployed on cloud platforms such as AWS, Azure, and Google Cloud with minimal configuration.

Another notable component of Apache Spark is Spark SQL, which enables users to run SQL queries on structured and semi-structured data. It provides seamless integration with databases, data lakes, and even JSON and Parquet files. Through the DataFrame API, Spark SQL allows users to interact with data using SQL-like commands while taking advantage of Spark's distributed computing capabilities.

PySpark, the Python API for Apache Spark, has played a significant role in making Spark more accessible to data scientists and engineers. Since Python is a widely used language in data analytics and machine learning, PySpark provides an intuitive way to harness Spark's power without requiring extensive knowledge of Scala or Java. PySpark's API allows users to manipulate large-scale data using familiar Python libraries such as Pandas, NumPy, and Scikit-learn. The DataFrame API in PySpark is particularly effective for structured data analysis, enabling SQL operations and machine learning workflows with minimal code modifications. Additionally, PySpark supports

interactive analysis using Jupyter Notebooks, making it ideal for exploratory data analysis and prototyping.

One of the most compelling applications of Apache Spark is its capability to train and deploy machine learning models at scale. Traditional machine learning tools often struggle with large datasets due to memory limitations, but Spark MLlib offers a distributed framework that allows training on massive datasets efficiently. Spark MLlib provides a comprehensive suite of algorithms for classification, regression, clustering, and recommendation systems. These algorithms are optimized to run in a distributed environment, leveraging Spark's parallel computing model to process massive volumes of data.

Feature engineering, an essential part of machine learning, is seamlessly handled in Spark through various transformation functions. The MLlib library provides utilities for feature extraction, standardization, and dimensionality reduction, allowing users to preprocess data effectively before feeding it into machine learning models. Additionally, Spark MLlib supports pipeline development, which helps streamline machine learning workflows by chaining multiple data processing and modeling steps together. This modular approach improves reproducibility and simplifies hyperparameter tuning.

Beyond conventional machine learning, Spark has also been increasingly integrated with deep learning frameworks such as TensorFlow and Keras. By using Spark's distributed capabilities, neural networks can be trained on large datasets without requiring high-end hardware on a single machine. With tools like TensorFlowOnSpark and Deep Learning Pipelines, Spark can efficiently scale deep learning models across clusters, making it feasible to train and fine-tune complex neural networks on big data.

Spark's role in real-time machine learning is another crucial aspect of its growing adoption. Through Spark Streaming, data can be ingested and processed in real time, allowing for predictive modeling on live data streams. This functionality is widely used in applications such as fraud detection, recommendation systems, and automated anomaly detection, where immediate insights are necessary. By integrating Spark Streaming with MLlib, organizations can deploy machine learning models that continuously learn and adapt to incoming data, providing dynamic and up-to-date predictions.

As data volumes continue to grow, Spark remains one of the most effective platforms for handling large-scale data analytics and machine learning. Its ability to perform fast, distributed computations makes it an essential tool for businesses, researchers, and engineers working with massive datasets. Whether used for exploratory data analysis, real-time analytics, or deep learning, Apache Spark provides a scalable and efficient solution for modern data science challenges.

What is pysaprk?

PySpark is the Python API for Apache Spark, providing a powerful and scalable solution for handling large-scale data processing and machine learning. It allows developers and data scientists to utilize Spark's distributed computing framework while writing code in Python, a widely-used language in the data science community. PySpark bridges the gap between big data processing and Python-based data analytics, enabling efficient, parallelized computations on massive datasets.

One of PySpark's most significant advantages is its DataFrame API, which is modeled after Pandas DataFrames but is designed to work in a distributed environment. PySpark DataFrames provide an abstraction that allows users to manipulate structured and semi-structured data using SQL-like operations while leveraging Spark's parallel processing capabilities. This makes it an ideal tool for querying and transforming large datasets with minimal effort. Furthermore, PySpark supports interoperability with databases, cloud storage systems, and data lakes, allowing seamless data integration and manipulation.

PySpark also provides direct access to Spark's core RDD (Resilient Distributed Dataset) structure, which is a fundamental component of Spark's distributed computing model. RDDs enable fault-tolerant, parallelized operations on large datasets, making them essential for low-level data transformations. Although DataFrames offer a more user-friendly approach for structured data, RDDs remain a powerful tool for custom data transformations and computations in PySpark.

Another essential feature of PySpark is its integration with Spark SQL, allowing users to write SQL queries directly in Python and run them on distributed datasets. This capability enables analysts and engineers to work with big data using familiar SQL syntax, eliminating the need to convert between different data formats or programming paradigms. With built-in optimizations such as Catalyst Optimizer, Spark SQL ensures efficient query execution and performance tuning, making data retrieval faster and more efficient.

Beyond data processing, PySpark is widely used in machine learning through its MLlib and ML modules. MLlib provides scalable implementations of machine learning algorithms such as classification, regression, clustering, and recommendation systems. The ML module in PySpark introduces a more structured approach to building machine learning pipelines, allowing users to streamline feature engineering, model training, and hyperparameter tuning. With these tools, PySpark enables the development of large-scale, production-ready machine learning models that can operate on massive datasets without requiring extensive computational resources.

PySpark's compatibility with popular Python libraries such as Pandas, NumPy, and Scikit-learn further enhances its capabilities. Users can easily transition between Pandas DataFrames and PySpark DataFrames, enabling a hybrid approach to data processing. This interoperability is

particularly useful for exploratory data analysis, where smaller subsets of data can be analyzed locally before being scaled up for distributed processing in PySpark.

Additionally, PySpark supports streaming data processing through Spark Streaming, allowing real-time data ingestion and analysis. This is particularly beneficial in applications such as fraud detection, real-time recommendation engines, and monitoring systems, where continuous data streams must be processed instantly. By leveraging Spark Streaming, PySpark enables users to build scalable, fault-tolerant applications that react to incoming data in real-time.

Overall, PySpark is a versatile and powerful tool that brings together the flexibility of Python and the scalability of Apache Spark. Whether working with structured data in DataFrames, performing complex transformations with RDDs, or building machine learning models on massive datasets, PySpark provides an efficient and scalable solution for modern data processing challenges. As data volumes continue to grow, PySpark remains an essential tool for data engineers, scientists, and analysts seeking to unlock the full potential of big data analytics.

Model Evaluation & Testing

Implementation

In this project, we will use Spark to train and test our machine learning model. Since four machine learning models will be trained and ultimately tested, their code content is the same except for the variables they have, and by explaining one of them, other details can be reached.

1. Importing Required Libraries

The script imports essential libraries:

- `os, shutil`: Handles file operations.
- `psycopg2`: Connects to a PostgreSQL database.
- `time, datetime`: Measures time and timestamps.
- `pyspark.sql`: Manages structured data with DataFrames.
- `pyspark.ml`: Implements machine learning models and transformations.

```
import os
import shutil
import psycopg2
import time
import datetime
from pyspark.sql import SparkSession
from pyspark.ml import Pipeline
from pyspark.sql import DataFrame
from pyspark.ml import PipelineModel
from pyspark.sql.functions import col
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
```

```
from pyspark.ml.classification import DecisionTreeClassifier
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

2. Spark Configuration and Prometheus Metrics

Spark Environment Settings

```
os.environ['SPARK_DRIVER_EXTRA_JAVA_OPTIONS'] = '--illegal-access=permit'
os.environ['SPARK_EXECUTOR_EXTRA_JAVA_OPTIONS'] = '--illegal-access=permit'
```

Configures Spark to **permit access to certain internal Java APIs**, preventing potential execution errors.

```
# Create SparkSession
spark = SparkSession.builder \
    .appName("RandomCSVReadAndModelTraining") \
    .getOrCreate()
```

3. File Paths and Database Configuration

```
# CSV file paths
csv_file_path = "/home/hamidreza/Documents/bigdata/data/csv_file1.csv"
model_output_path = "/home/hamidreza/Documents/bigdata/model/model_s1"
csv_file_path_test = "/home/hamidreza/Documents/bigdata/data/csv_file2.csv"
model_output_path_test =
"/home/hamidreza/Documents/bigdata/model/model_s1/final_model"
```

Defines paths for training and testing datasets and model storage.

4. Database Connection

```
db_config = {
    "host": "localhost",
    "database": "mydb",
```

```

    "user": "hamid",
    "password": "1234",
    "port": 5432
}

```

5. Data Insertion and Connection Functions

```

def insert_data(cursor, data):
    insert_query = "INSERT INTO accuracy_data_model1 (rmse, timestamp, time) VALUES (%s, %s, %s)"
    cursor.execute(insert_query, data)

def connect_to_db():
    return psycopg2.connect(**db_config)

```

6. Read Random Data from CSV

```

def read_random_lines(csv_file_path: str, num_samples: int = 100) -> DataFrame:
    df = spark.read.csv(csv_file_path, header=True, schema=schema)
    fraction = 600/df.count()
    sampled_df = df.sample(withReplacement=False, fraction=fraction, seed=42)
    return sampled_df

```

7. Model Training Function

```

def train_model(df: DataFrame):
    # Convert numeric columns to DoubleType
    numeric_cols = ["DepDelay", "AirTime", "Distance"]

    df = df.select(
        *[col(c).cast(DoubleType()).alias(c) if c in numeric_cols + ["ArrDelay"] else col(c) for c in df.columns]
    )

```

8. Converts numeric columns to `DoubleType`.

```
    categorical_cols = ["Airlines", "OriginCityName", "DestCityName", "Month_Str",
"DayOfWeek_Str"]
    indexers = []
    encoders = []
    encoded_cols = []

    for c in categorical_cols:
        distinct_values = df.select(c).distinct().count()
        if distinct_values > 1:
            indexer = StringIndexer(inputCol=c,
outputCol=f"{c}_index",handleInvalid='skip')
            encoder = OneHotEncoder(inputCol=f"{c}_index", outputCol=f"{c}_encoded")

            indexers.append(indexer)
            encoders.append(encoder)
            encoded_cols.append(f"{c}_encoded")
```

9. Prepares categorical variables for encoding.

```
df = df.na.fill({"Airlines": "Unknown", "OriginCityName": "Unknown",
"DestCityName": "Unknown"})
```

10. Defines a Random Forest Regressor model to predict arrival delay (`ArrDelay`).

```
pipeline = Pipeline(stages=indexers + encoders + [assembler, rf])
model = pipeline.fit(df)
```

11. Model Testing Function

```
def test_model():  
    conn = connect_to_db()  
    cursor = conn.cursor()  
    model = PipelineModel.load(model_output_path_test)
```

12. Loads the saved model.

```
df = spark.read.csv(csv_file_path, header=True, schema=schema)  
df.count()
```

13. Reads the test dataset

```
evaluator = RegressionEvaluator(labelCol="ArrDelay", predictionCol="prediction",  
metricName="rmse")
```

14. Evaluates the model performance using RMSE

```
rmse = evaluator.evaluate(test_df)  
print(f"✅ rmse: {rmse:.2f}%")
```

15. save to database

```
test_df.select("Index", "ArrDelay", "prediction").show()  
  
current_datetime = datetime.datetime.now()  
formatted_datetime = current_datetime.strftime("%Y-%m-%d %H:%M:%S")  
accuracy_data = ()  
accuracy_data = (1-rmse , time.time(),formatted_datetime)  
insert_data(cursor, accuracy_data)  
conn.commit()
```

16. Running the Training and Testing Loop

```
while True:  
    df_sample = read_random_lines(csv_file_path)  
    train_model(df_sample)
```

```
#  
test_model()
```

Continuously trains and tests the model on new data.

Review

The provided script is a machine learning pipeline built using PySpark, PostgreSQL, and Prometheus for monitoring. It automates the process of reading data from CSV files, preprocessing features, training a machine learning model, evaluating its performance, and storing the results in a PostgreSQL database. The model used is a `RandomForestRegressor`, which predicts arrival delay (`ArrDelay`) based on various flight-related features. The pipeline is structured to run continuously, repeatedly training and testing new models on different data samples.

At the start, necessary libraries are imported, including PySpark for distributed data processing, `psycpg2` for database interaction, and Prometheus client for tracking performance metrics. A Spark session is created to enable parallel computation, and Prometheus metrics are initialized to monitor the number of trained and tested models, accuracy trends, and model training time.

The script defines paths for CSV files containing training and test datasets, as well as directories for saving trained models. A PostgreSQL configuration dictionary stores connection details for inserting model evaluation results into a dedicated database table. A Prometheus HTTP server is also started to expose monitoring data.

Feature engineering involves converting numerical columns to `DoubleType` and encoding categorical variables using `StringIndexer` and `OneHotEncoder`. A `VectorAssembler` is used to combine all transformed features into a single vector column, which serves as input to the machine learning model. The `RandomForestRegressor` is trained on this processed data, capturing relationships between features and the target variable (`ArrDelay`).

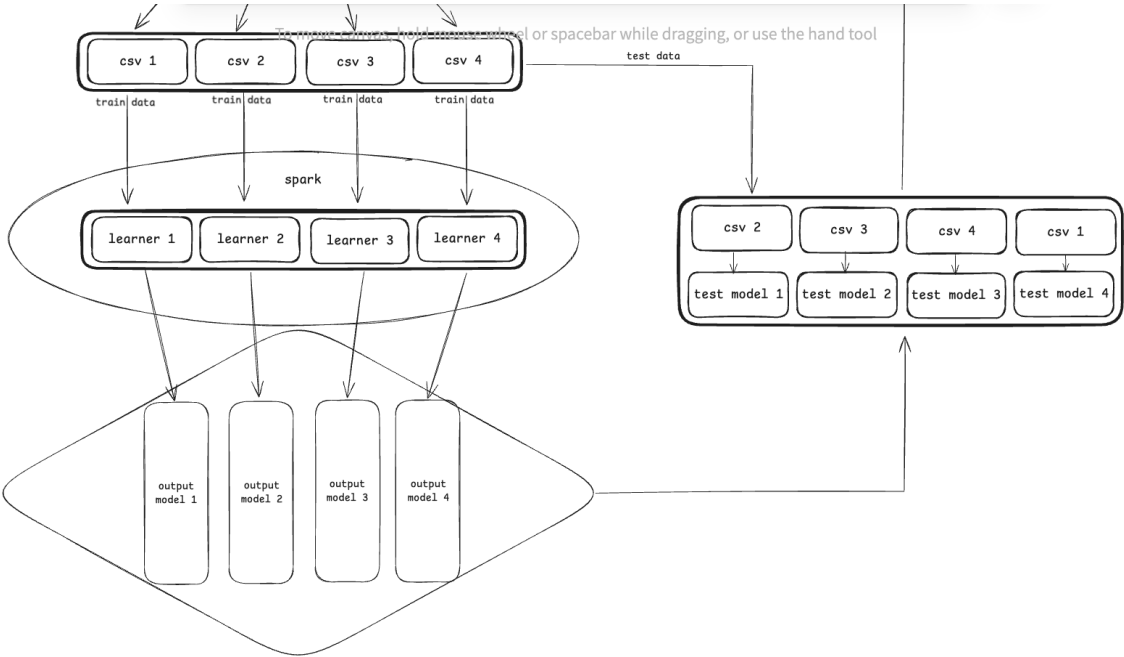
The trained model is saved to a specified directory, allowing it to be reloaded later for evaluation. During testing, the saved model is applied to new data, predictions are generated, and performance is assessed using the root mean square error (RMSE). The script logs evaluation results in PostgreSQL, ensuring that performance trends are recorded for future analysis.

The entire process is automated in a continuous loop, where the script reads a random subset of data, trains a new model, tests it, and logs the results. This approach allows for iterative model improvements and ensures that the model remains up-to-date with the latest data. The integration of Prometheus provides real-time insights into model performance, making it easier to track

improvements or detect issues over time. This setup is particularly useful for applications requiring frequent retraining, such as flight delay predictions, where new data can help refine the model's accuracy.

Below are some examples of Grafana images showing our user panels and graphs. Our models have some differences. The parameters we used in these models are as shown in the table below. According to this, we will have the appropriate output.

	MODEL1	MODEL2	MODEL3	MODEL4
NUMTREE	100	150	100	100
MAXDETH	7	7	10	7
MIN INSTANCE PER NODE	1	1	1	2



Monitoring & Visualization with Prometheus & Grafana

Grafana: An Overview

Grafana is an open-source **visualization and monitoring tool** that enables users to create interactive and real-time dashboards from various data sources. It supports multiple backend data storage systems such as **Prometheus, InfluxDB, MySQL, PostgreSQL, and Elasticsearch**. Grafana is widely used for **observability, performance monitoring, and analytics** across different industries, including IT infrastructure, application monitoring, and business intelligence.

One of Grafana's key strengths is its ability to provide **dynamic and customizable dashboards**, allowing users to visualize data trends, set up alerts, and share insights with teams. It also supports **real-time querying**, enabling users to interact with data efficiently. With its user-friendly interface and flexible plugin system, Grafana can integrate with a wide range of technologies, making it a powerful choice for data visualization.

Prometheus: An Overview

Prometheus is an open-source **monitoring and alerting system** designed specifically for high-dimensional time-series data. Initially developed by **SoundCloud**, it has become a standard in cloud-native monitoring due to its **scalability, efficiency, and deep integration with Kubernetes and containerized environments**.

Prometheus operates by **scraping metrics from configured targets**, storing the data in a **time-series database**, and allowing users to query the data using its built-in **PromQL (Prometheus Query Language)**. It also supports **multi-dimensional data modeling**, enabling advanced analysis based on labels and filtering. Prometheus is designed to work independently and efficiently without requiring external storage or dependencies. It also includes a built-in alerting mechanism called **Alertmanager**, which sends notifications based on predefined conditions.

Connecting Grafana and Prometheus

Connecting Grafana with Prometheus allows users to **visualize Prometheus metrics in real-time dashboards**. Prometheus collects and stores time-series data from applications, infrastructure, or services, while Grafana provides a **graphical interface** to analyze and interpret this data.

By integrating Grafana with Prometheus, users can **create insightful dashboards to monitor performance metrics, system health, and application behavior**. This connection is highly beneficial for **detecting anomalies, troubleshooting issues, and making data-driven decisions**. The setup allows organizations to track trends over time, optimize resource usage, and **set up proactive alerting mechanisms** to respond to system failures before they escalate.

This integration is widely used in DevOps, cloud monitoring, and big data applications where real-time observability is crucial. By leveraging both tools, teams can **enhance monitoring efficiency, reduce downtime, and improve overall system reliability**. In the following, we will explain the parameters defined in the metrics step by step.

First stage metrics → 2__producer_from_datasource.py

```
MESSAGE_COUNT = Counter('kafka_message_count_produce_A', 'Number of messages sent to Kafka')
MESSAGE_LATENCY = Histogram('kafka_message_latency_seconds_producer_A', 'Time taken to send messages to Kafka')
```

The provided code snippet defines two Prometheus metrics for monitoring Kafka message production. The first metric, `MESSAGE_COUNT`, is a **Prometheus Counter**, which continuously increments over time. It keeps track of the total number of messages sent to Kafka by a producer, ensuring that message production is occurring as expected. The metric is labeled as `'kafka_message_count_produce_A'`, indicating that it is specific to **Producer A**.

The second metric, `MESSAGE_LATENCY`, is a **Prometheus Histogram** that measures the latency of sending messages to Kafka. This metric helps in understanding how long it takes for messages to be produced, capturing variations in send times and potential bottlenecks. The label `'kafka_message_latency_seconds_producer_A'` specifies that it tracks the latency for **Producer A**. Since histograms record distributions of values, they are useful for identifying performance issues and optimizing message throughput.

These metrics are essential for monitoring Kafka producers in real-time. By integrating them into **Grafana**, teams can visualize message production trends, identify latency spikes, and detect anomalies that could indicate failures or slowdowns. Tracking the total count of messages ensures that Kafka is receiving data as expected, while latency monitoring helps in diagnosing potential network or system delays. These insights enable proactive system optimization, preventing issues before they impact downstream consumers.

Second stage metrics → 3__consumer_save_db.py

```
# Prometheus metrics
MESSAGE_CONSUMED_COUNT = Counter('kafka_message_consumed_count_save_db', 'Number of
messages consumed from Kafka')
INSERT_BATCH_COUNT = Counter('kafka_insert_batch_count_save_db', 'Number of data
batches inserted into the database')
ERROR_COUNT = Counter('kafka_consumer_error_count_save_db', 'Number of errors
encountered by the Kafka consumer')
MESSAGE_PROCESSING_LATENCY =
Histogram('kafka_message_processing_latency_seconds_save_db', 'Time taken to process
a message')
```

This code defines **Prometheus metrics** to monitor the performance and reliability of a **Kafka consumer** that processes messages and stores them in a database. The first metric, `MESSAGE_CONSUMED_COUNT`, is a **Prometheus Counter** that tracks the total number of messages successfully consumed from Kafka. This helps in monitoring whether the consumer is actively receiving data and can be used to identify drops in message consumption.

The `INSERT_BATCH_COUNT` metric is another **Counter** that records the number of data batches successfully inserted into the database. Since messages are often processed in batches, this metric provides insights into how frequently and efficiently the system is committing processed data to storage. Monitoring this can help detect database bottlenecks or failures in batch processing.

The `ERROR_COUNT` metric keeps track of errors encountered by the Kafka consumer. This is crucial for identifying issues related to **message deserialization, network failures, or database insert errors**. A sudden spike in this counter can indicate problems with the data pipeline that require immediate attention.

The `MESSAGE_PROCESSING_LATENCY` metric is a **Prometheus Histogram** that measures the time taken to process each message. By analyzing this histogram, teams can understand how long it takes to handle Kafka messages, detect performance degradation, and optimize processing times. A significant increase in processing latency could indicate computational inefficiencies, increased load, or delays in writing to the database.

Together, these metrics provide a **comprehensive view of Kafka consumer performance**, helping ensure **smooth message consumption, efficient batch insertion, low error rates, and optimal processing times**. By integrating these metrics into **Grafana**, teams can visualize trends, set alerts for anomalies, and take proactive measures to maintain system stability.

Third stage metrics → `4__managers.py`

```
# Prometheus metrics
MESSAGE_SENT_COUNT = Counter('kafka_message_sent_count_manager', 'Number of messages
sent to Kafka manager')
RECORDS_FETCHED_COUNT = Counter('db_records_fetched_count_manager', 'Number of
records fetched from database manager')
BATCH_PROCESSING_LATENCY = Histogram('db_batch_processing_latency_seconds_manager',
'Time taken to process a batch of records manager')
```

This code defines **Prometheus metrics** to monitor the interaction between a database and Kafka, particularly focusing on data retrieval, message transmission, and processing efficiency. The first metric, `MESSAGE_SENT_COUNT`, is a **Prometheus Counter** that tracks the number of messages sent from the system to a Kafka manager. Monitoring this metric helps ensure that messages are being transmitted consistently and allows detection of any unexpected drops in Kafka message production.

The second metric, `RECORDS_FETCHED_COUNT`, is another **Counter** that records the total number of records retrieved from the database manager. This provides visibility into how often and how many records are being fetched from storage for processing. A decrease in this count could indicate database access issues or inefficiencies in data retrieval.

The third metric, `BATCH_PROCESSING_LATENCY`, is a **Histogram** that measures the time taken to process a batch of records before they are sent to Kafka. This helps in assessing the efficiency of the data pipeline, identifying potential slowdowns, and optimizing batch processing performance. If this latency metric increases, it could indicate **computational inefficiencies, network delays, or database slowdowns** affecting the system.

These metrics collectively provide insights into **Kafka message flow, database retrieval rates, and processing efficiency**, enabling teams to monitor system health, detect anomalies, and optimize data pipeline performance. By integrating them into **Grafana**, teams can track trends in real-time, set up alerts for abnormal behavior, and ensure a stable and efficient data processing workflow.

Third stage metrics → 5__consumer_spark.py

```
MESSAGE_CONSUMED_COUNT = Counter('kafka_message_consumed_count', 'Number of messages consumed from Kafka', ['topic'])
ROW_WRITTEN_COUNT = Counter('csv_row_written_count', 'Number of rows written to CSV files', ['file'])
```

This code defines **Prometheus metrics** to monitor the flow of data from Kafka into CSV files, ensuring that messages are properly consumed and stored. The first metric, `MESSAGE_CONSUMED_COUNT`, is a **Prometheus Counter** that tracks the total number of messages consumed from Kafka. This counter includes a label, `['topic']`, allowing messages to be categorized based on their Kafka topic. This helps in analyzing message consumption across different topics, detecting imbalances, and ensuring all data streams are being processed correctly.

The second metric, `ROW_WRITTEN_COUNT`, is another **Counter** that records the number of rows written to CSV files. It includes a label, `['file']`, which enables tracking the number of records written to specific CSV files. This is particularly useful when multiple files are being written simultaneously, as it helps monitor data distribution across different output files and detect potential storage or processing bottlenecks.

By tracking these metrics, teams can ensure that Kafka consumers are **processing messages efficiently** and that data is being **persisted correctly** in CSV files. If message consumption slows down or row writing becomes inconsistent, these metrics can help identify issues such as Kafka consumer lag, file system limitations, or data pipeline inefficiencies. When integrated into **Grafana**, these metrics allow for real-time monitoring, visualization, and alerting, enabling proactive issue resolution and smooth data pipeline operations.

Fifth stage metrics → 6_1__learning_csv1.py

```
MODEL_TRAINED_COUNT = Counter('model_trained_count_line_1', 'Number of models trained  
line 1')  
MODEL_TESTED_COUNT = Counter('model_tested_count_line_1', 'Number of models tested  
line 1')  
MODEL_TRAINING_TIME = Histogram('model_training_time_seconds_line_1', 'Model training  
time in seconds line 1')
```

This code defines **Prometheus metrics** for monitoring the training and testing of machine learning models. The `MODEL_TRAINED_COUNT` metric is a **Counter** that tracks the total number of models trained. Each time a model is successfully trained, this counter increments, allowing monitoring of training frequency and detecting any unexpected drops in training activity.

The `MODEL_TESTED_COUNT` metric is another **Counter** that records the number of models tested. This helps ensure that models are not only being trained but also evaluated on test data. A discrepancy between trained and tested models may indicate that some models are not being validated properly.

The `MODEL_TRAINING_TIME` metric is another **Histogram** that measures the time taken to train each model. This provides insights into computational efficiency and helps detect slowdowns in the training process. A significant increase in training time might indicate hardware issues, increased data complexity, or inefficient algorithm tuning.

By integrating these metrics into **Grafana**, teams can visualize model training and testing trends, set alerts for anomalies, and optimize machine learning workflows. This setup enables real-time monitoring of model performance, ensuring that the training pipeline remains efficient and effective.

Fifth stage metrics → 6_2__learning_csv1.py

```
MODEL_TRAINED_COUNT = Counter('model_trained_count_line_2', 'Number of models trained  
line 2')  
MODEL_TESTED_COUNT = Counter('model_tested_count_line_2', 'Number of models tested  
line 2')  
MODEL_TRAINING_TIME = Histogram('model_training_time_seconds_line_2', 'Model training  
time in seconds line 2')
```

This code defines **Prometheus metrics** to monitor the training and testing process of machine learning models in a specific pipeline referred to as **line 2**. These metrics help track the number of trained and tested models, their accuracy over time, and the duration required for training.

The `MODEL_TRAINED_COUNT` metric is a **Counter** that keeps track of the total number of models trained. Each time a new model is successfully trained, this counter increases. Monitoring this helps in identifying whether the model training pipeline is running as expected and whether new models are being generated at the expected rate.

The `MODEL_TESTED_COUNT` metric is another **Counter** that records the number of models tested. This metric ensures that models are not just being trained but also evaluated for their performance. If the number of tested models is significantly lower than trained models, it could indicate an issue where models are not being properly validated.

The `MODEL_TRAINING_TIME` metric is another **Histogram** that captures the duration required to train each model. Analyzing training times helps in optimizing computational resources and detecting slowdowns that might arise from increased dataset size or inefficient hyperparameter configurations.

Integrating these metrics into **Grafana** allows real-time visualization, enabling teams to monitor model performance, detect anomalies, and fine-tune machine learning workflows efficiently. This setup ensures continuous improvement of models while maintaining reliability in the pipeline.

Fifth stage metrics → 6_3__learning_csv1.py

```
MODEL_TRAINED_COUNT = Counter('model_trained_count_line_3', 'Number of models trained  
line 3')  
MODEL_TESTED_COUNT = Counter('model_tested_count_line_3', 'Number of models tested  
line 3')  
MODEL_TRAINING_TIME = Histogram('model_training_time_seconds_line_3', 'Model training  
time in seconds line 3')
```

This code defines **Prometheus metrics** to monitor the machine learning model training and evaluation process for a specific **pipeline labeled as line 3**. These metrics provide insights into the frequency of model training and testing, accuracy trends over time, and the computational efficiency of the training process.

The `MODEL_TRAINED_COUNT` metric is a **Counter** that keeps track of the total number of models trained. Every time a model completes training, this counter increases, helping in monitoring how often models are being generated and ensuring the pipeline is running as expected.

The `MODEL_TESTED_COUNT` metric is another **Counter** that records the number of models tested. This metric ensures that models are not only being trained but also validated for their effectiveness. If there is a mismatch between trained and tested models, it may indicate that some models are not undergoing proper evaluation, which can lead to unreliable results.

Integrating these metrics with **Grafana** enables real-time monitoring, visualization, and alerting, ensuring continuous improvement of machine learning models while maintaining system reliability. This setup is essential for large-scale machine learning workflows where performance tracking and optimization are crucial for maintaining high model accuracy and efficiency.

Fifth stage metrics → 6_4__learning_csv1.py

```
MODEL_TRAINED_COUNT = Counter('model_trained_count_line_4', 'Number of models trained  
line 4')  
MODEL_TESTED_COUNT = Counter('model_tested_count_line_4', 'Number of models tested  
line 4')  
MODEL_TRAINING_TIME = Histogram('model_training_time_seconds_line_4', 'Model training  
time in seconds line 4')
```

This code defines **Prometheus metrics** to monitor the training and testing process of machine learning models within a specific pipeline labeled as **line 4**. These metrics track the number of trained and tested models, evaluate model accuracy over time, and measure training duration to ensure efficiency and reliability in the machine learning workflow.

The `MODEL_TRAINED_COUNT` metric is a **Counter** that tracks the total number of models successfully trained. Every time a model completes training, this counter increments, providing a way to monitor how frequently models are being generated. A steady increase indicates an active training pipeline, while a sudden drop may signal issues in the training process.

The `MODEL_TESTED_COUNT` metric is another **Counter** that records the number of models evaluated after training. Ensuring that every trained model is tested is crucial for assessing performance before deployment. If this count falls behind the trained model count, it could indicate missing validation steps or issues in the testing process.

The `MODEL_TRAINING_TIME` metric is another **Histogram** that measures the time required to train each model. By analyzing training time variations, this metric helps optimize computational resources and detect slowdowns. Longer training times may indicate increased dataset complexity, inefficient algorithms, or insufficient computational resources.

Integrating these metrics with **Grafana** enables real-time monitoring and visualization, allowing teams to track model training efficiency, detect performance anomalies, and make data-driven optimizations. This setup is particularly valuable in large-scale machine learning pipelines where continuous monitoring is essential for maintaining model reliability and performance.

Below are some sample images from Grafana showing our user panels and charts.

